AUTOM-97 LAB: Monaco Configuration as Code

> **Series:** AUTOM — Dynatrace Automation | **Reference:** 97 — Monaco Hands-On LAB | **Created:** April 2026 | **Last Updated:** 10/24/2026

Overview

Hands-on lab for installing Monaco CLI, downloading existing Dynatrace configuration, modifying it, validating, and deploying to a target environment. Includes CI/CD pipeline integration with GitHub Actions. Complete the lecture notebook **AUTOM-03: Monaco Configuration-as-Code** first, then work through these exercises step by step.

Table of Contents

1. Install Monaco CLI
2. Create Access Token
3. Create Manifest File
4. Download Existing Configuration
5. Explore Downloaded Configuration
6. Create a New Configuration
7. Validate Configuration
8. Deploy Configuration
9. Delete Configuration
10. Multi-Environment Promotion
11. GitHub Actions CI/CD Pipeline
12. Summary & Checklist

Prerequisites

Requirement	Details
Completed	AUTOM-03: Monaco Configuration-as-Code (lecture notebook)
Dynatrace Environment	SaaS tenant (Grail enabled)
Permissions	Ability to create an access token in the environment and a platform token in your account
Local Tools	Terminal with `curl` and `chmod` (Monaco has no Homebrew formula)
Git	Git CLI installed (for CI/CD section)
GitHub Account	Required for Section 11 (CI/CD pipeline)

1. Install Monaco CLI

Monaco is a standalone binary with no runtime dependencies. Choose the release asset for your platform — Monaco is distributed only as GitHub release binaries (there is no Homebrew formula).

macOS (Apple silicon)

```
# Download the latest Monaco binary for macOS (Apple silicon)
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-darwin-arm64 -o monaco
chmod +x monaco
sudo mv monaco /usr/local/bin/
# Intel Macs: use monaco-darwin-amd64
```

Linux (x86-64)

```
# Download the latest Monaco binary for Linux (amd64)
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco

# Make it executable and move to PATH
chmod +x monaco
sudo mv monaco /usr/local/bin/
# Other Linux architectures: monaco-linux-arm64, monaco-linux-386
```

Windows (PowerShell)

```
# Download Monaco for Windows
Invoke-WebRequest -Uri "https://github.com/Dynatrace/dynatrace-configuration-
as-code/releases/latest/download/monaco-windows-amd64.exe" -OutFile
"monaco.exe"

# Move to a directory in your PATH
Move-Item monaco.exe C:\Tools\monaco.exe
```

Verify Installation

```
# Verify Monaco is installed and check version
monaco version
```

> **Note:** Monaco releases are available at github.com/Dynatrace/dynatrace-configuration-as-code/releases. Check for the latest version before installing.

---

2. Create Access Token

This LAB works on a **service-level objective** (`type: slo-v2`), a platform configuration type, so it needs a **platform token** as well as a classic access token. Once the manifest has a platform token, Monaco sends Settings 2.0 through it too — the access token is then used for the classic Configuration APIs.

Platform Token Scopes (required for this LAB)

Scope	Purpose
`app-engine:apps:run`	Platform metadata lookup Monaco runs before any download or deploy
`settings:objects:read`	Download Settings 2.0 objects
`settings:objects:write`	Deploy Settings 2.0 objects
`settings:schemas:read`	Read Settings 2.0 schemas
`slo:slos:read`	Download SLOs (`--only-slo-v2`)
`slo:slos:write`	Deploy and delete SLOs
`slo:objective-templates:read`	Listed for `slo-v2` on the Monaco access-handling page

Add the scopes for the other platform types only if the project will manage them — workflows, documents, segments and buckets each need their own scopes (read/write, plus delete for buckets, documents and segments); the Monaco access-handling page lists them per type. Create the token under **My platform tokens** (`https://myaccount.dynatrace.com/platformTokens`), ideally on a service user (AUTOM-04 §3).

Checking the deployed SLO's evaluation in the app is done as *you*, so your own user needs read access to the data its SLI queries.

> **Sources:** Monaco manifest and resources (DT docs) — `app-engine:apps:run` : *"This permission is required to access Dynatrace Platform metadata endpoints."*; Monaco API support and access handling (DT docs) — `slo-v2` row: `slo:slos:read` , `slo:slos:write` , `slo:objective-templates:read` ; Platform tokens (DT docs) — *"This opens https://myaccount.dynatrace.com/platformTokens"*; client set (Dynatrace GitHub). **Derived:** Settings 2.0 through the platform token is read from that file at v2.30.0, which builds the classic settings client only when no platform credential is configured.

Access Token Scopes (classic Configuration APIs; Settings 2.0 only when no platform credential is configured)

Scope	Purpose
`Access problem and event feed, metrics, and topology` (`DataExport`)	Always required by Monaco
`Read configuration` (`ReadConfig`)	Download classic API configs
`Write configuration` (`WriteConfig`)	Deploy classic API configs
`Read settings` (`settings.read`)	Settings 2.0 downloads in a manifest **without** a platform token
`Write settings` (`settings.write`)	Settings 2.0 deploys in a manifest **without** a platform token
`Read SLO` (`slo.read`)	*Classic* SLOs only — not needed for `slo-v2`
`Write SLO` (`slo.write`)	*Classic* SLOs only — not needed for `slo-v2`
`Read entities` (`entities.read`)	Entity queries during download
`Create and read synthetic monitors, locations, and nodes` (`ExternalSyntheticIntegration`)	Synthetic config management

Create the token in the environment under **Access Tokens** with the scopes above.

> **Sources:** Monaco manifest and resources (DT docs) — *"Access tokens for the Dynatrace Monaco CLI always require at least the Access problem and event feed, metrics, and topology"*

Set Environment Variables

```
# Set your Dynatrace environment URL and tokens
# Replace with your actual values. Use the PLATFORM URL (.apps.), not the
classic .live. URL:
# with a platform token in the manifest, Monaco looks up the classic URL from
it.
export DT_ENV_URL="https://&amp;lt;your-environment-id&amp;gt;.apps.dynatrace.com"
export
DT_API_TOKEN="dt0c01.XXXXXXXX.YYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYYY
YYYYYYYYYYYYYYYY"
# Platform token for SLOs, workflows, documents, buckets and segments
(manifest auth.platformToken)
# Required for this LAB — the exercises use type: slo-v2
export DT_PLATFORM_TOKEN="&amp;lt;your-platform-token&amp;gt;"

# Verify the variables are set
echo "Environment: $DT_ENV_URL"
echo "Token set: $([ -n "$DT_API_TOKEN" ] &amp;amp;&amp;amp; echo yes || echo no)"
echo "Platform token set: $([ -n "$DT_PLATFORM_TOKEN" ] &amp;amp;&amp;amp; echo yes ||
echo no)"
```

> **Warning:** Never commit API tokens to version control. Use environment variables or a secrets manager.
>
> **If the URL is wrong:** a `.live.` URL with a platform token fails on the first command with *"make sure you are using a Dynatrace Platform URL and not a Dynatrace Classic URL when authenticating with an OAuth client or platform token"*; a missing `app-engine:apps:run` scope fails with *"oAuth client or platform token may be missing required scope 'app-engine:apps:run'"*.
>
> **Sources:** metadata client (Dynatrace GitHub) — both error messages, v2.30.0.

---

3. Create Manifest File

The `manifest.yaml` is the entry point for every Monaco project. It defines which projects to manage and which environments to target.

Project Directory Setup

The cell below creates `dynatrace-config/` and changes into it. **Run every later command from inside `dynatrace-config/`** — the paths in the rest of this LAB are relative to it.

```
# Create the project directory structure
mkdir -p dynatrace-config/my-project
cd dynatrace-config
```

Manifest Format

Create `dynatrace-config/manifest.yaml` with this content:

```
manifestVersion: "1.0"

projects:
  - name: my-project
    path: my-project

environmentGroups:
  - name: default
    environments:
      - name: dev
        url:
          type: environment
          value: DT_ENV_URL
        auth:
          token:
            name: DT_API_TOKEN
          platformToken:
            name: DT_PLATFORM_TOKEN
```

Field Reference

Field	Description
`manifestVersion`	Always `"1.0"` for current Monaco
`projects[].name`	Logical name for the project
`projects[].path`	Relative path to the project directory

Field	Description
`environmentGroups[].name`	Group name (used for overrides)
`environments[].name`	Environment identifier (used with `--environment` flag)
`environments[].url.value`	Environment variable name containing the tenant URL — the platform (`.apps.`) URL when `platformToken` or `oAuth` is set
`environments[].auth.token.name`	Environment variable name containing the API (access) token — settings and classic configuration APIs
`environments[].auth.platformToken.name`	Environment variable name containing a platform token (Monaco 2.24.0+) — needed for workflows, documents, Grail buckets and segments. An `oAuth` client (`clientId.name` / `clientSecret.name`) is the alternative; you cannot define both

> **Important:** The `url.value` and `auth.token.name` fields reference **environment variable names**, not the actual URL or token values. Monaco reads the values from your shell environment at runtime. Access tokens and platform tokens are not interchangeable — with only `token` defined, `monaco download` returns Settings 2.0 and classic configurations but none of the platform types listed in §4 — including the SLOs this LAB works on.

---

4. Download Existing Configuration

Downloading is the fastest way to get started. Monaco exports your current tenant configuration into versioned YAML and JSON files.

Download All Configuration

```
# Download all configuration from the dev environment
monaco download --manifest manifest.yaml --environment dev --output-folder
downloaded
```

Download Specific Configuration Types

```
# Download only SLOs and segments (platform types — needs auth.platformToken)
monaco download --manifest manifest.yaml --environment dev --output-folder
downloaded \
  --only-slo-v2 --only-segments

# Classic inventory, while the tenant still has them: specific Settings 2.0
schemas
# monaco download --manifest manifest.yaml --environment dev --output-folder
downloaded \
#   --settings-schema builtin:alerting.profile,builtin:management-zones
```

What Gets Downloaded

Monaco organizes downloaded configuration by type:

Config type	Description	Examples
`slo-v2`	SLOs on the modern SLO app	SLO Service API objects
`segment`	Filter segments	Segment definitions
`automation`	Workflows, business calendars, scheduling rules	Automation API objects
`document`	Dashboards, notebooks, launchpads	Document API objects
`bucket`	Grail buckets	Bucket definitions

Config type	Description	Examples
`settings`	Settings 2.0 objects	`builtin:davis.anomaly-detectors`, `builtin:ownership.teams`; classic estates also `builtin:alerting.profile`, `builtin:management-zones`
`api` (internally `classic`)	Classic Configuration APIs	`type: {api: request-attributes}` — request attributes, classic dashboards

The type names are the `type:` fields from the Monaco type-fields reference (DT docs); a classic config is written `type: {api: <name>}`, not `type: classic`. The matching `--only-*` download flags are `--only-slo-v2`, `--only-segments`, `--only-automation`, `--only-documents`, `--only-buckets`, `--only-settings`, `--only-apis` (and, up to v2.30.x, `--only-openpipeline`), and can be combined. Monaco 2.31.0 (10/23/2026) removed the deprecated `openpipeline` type — the release notes (Dynatrace GitHub): *"Removed support for deprecated OpenPipeline configurations (type `openpipeline`)"* — and its download command no longer has `--only-openpipeline`. OpenPipeline configuration is Settings 2.0 objects, so download it with `--only-settings` or `--settings-schema` with each `builtin:openpipeline.*` schema in use, plus `--admin-access` for pipelines other users own.

> **Platform types need platform auth.** `slo-v2`, `document`, `automation`, `bucket` and `segment` configs come through the Platform APIs, so they appear only when the manifest environment has `auth.platformToken` or `auth.oAuth` (see §3).
>
> **Note:** Some classic API types (`aws-credentials`, `azure-credentials`, `kubernetes-credentials`, `credential-vault`, `extension`) support deploy but **not** download — the Monaco API support and access handling (DT docs) page marks each *"It can't be downloaded."*

---

5. Explore Downloaded Configuration

After downloading, examine the directory structure and understand the file format.

Directory Structure

```
# View the downloaded directory structure
find downloaded -type f | head -30
```

Expected output looks like:

```
downloaded/
  manifest.yaml
  project_dev/
    slo-v2/
      config.yaml
      &lt;template&gt;.json
    segment/
      config.yaml
      &lt;template&gt;.json
    ...
```

A manifest download writes a project named `project` with the environment appended — `project_dev` here — unless you pass `--project <name>` (and `--force` to stop the environment suffix). Folder names follow the config type, stripped of characters other than letters, digits, `-`, `_` and `.`; a Settings 2.0 download produces one folder per schema, so `builtin:management-zones` lands in `builtinmanagement-zones/`. Check your own download rather than relying on the exact names here.

> **Sources:** Monaco commands reference (DT docs) — *"By default, the project name is project ."*; download command (Dynatrace GitHub) — `--force` : *"Manifest download: Never append the source environment name to the project folder name."*; name sanitizer (Dynatrace GitHub).

Example config.yaml

Each config type directory contains a `config.yaml` that references JSON template files:

```
configs:
  - id: checkout-availability
    config:
      name: "Checkout availability"
      template: checkout-availability.json
    type: slo-v2
```

Key Fields

Field	Description
`id`	Unique identifier for this config within the project
`config.name`	Display name in the Dynatrace UI
`config.template`	Path to the JSON payload file
`type`	The config type — `slo-v2`, `segment`, `{automation: {resource: workflow}}`, `{document: {kind: dashboard}}`, …
`type.settings.schema` / `type.settings.scope`	For Settings 2.0 configs only: the schema ID and the deployment scope (`environment`, `host`, `service`, …)

Examine a JSON Template

```
# View the first downloaded SLO template
f=$(find downloaded/project_dev/slo-v2 -name '*.json' | head -1)
python3 -m json.tool "$f" | head -30
```

> **Tip:** The JSON templates contain the exact payload that Monaco sends to the Dynatrace API. You can modify these files to change configuration values.

---

6. Create a New Configuration

Create a new **SLO** from scratch to understand how Monaco configurations are authored. An SLO is a good first target: it is a single self-contained payload, it changes nothing about how your tenant alerts, and you can see it evaluate in the SLO app minutes after deploying.

> **Classic equivalent — earlier versions of this LAB** created an alerting profile here. That still works on a tenant that has not been upgraded, but

> `builtin:alerting.profile` is on Dynatrace's list of Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs); its successor is a problem-triggered workflow (`type: {automation: {resource: workflow}}`). The classic walkthrough is kept as a labelled block at the end of this section.

Step 1: Create the Directory

```
# Create directory for SLO configs
mkdir -p my-project/slo-v2   # run inside dynatrace-config/
```

Step 2: Create config.yaml

Create `dynatrace-config/my-project/slo-v2/config.yaml`:

```
configs:
  - id: checkout-availability
    config:
      name: "Checkout availability"
      template: checkout-availability.json
      parameters:
        slo_target: 99.0
    type: slo-v2
```

Step 3: Create the JSON Template

Create `dynatrace-config/my-project/slo-v2/checkout-availability.json`:

```
{
  "name": "{{ .name }}",
  "description": "Request success ratio, rolling 7 days — Monaco LAB",
  "tags": ["lab:monaco"],
  "criteria": [
    {
      "target": {{ .slo_target }},
      "warning": 99.5,
      "timeframeFrom": "now-7d",
      "timeframeTo": "now"
    }
  ],
  "customSli": {
    "indicator": "timeseries { total = sum(dt.service.request.count), failures
= sum(dt.service.request.failure_count) }\n| fieldsAdd sli = ((total[] -
failures[]) / total[]) * 100\n| fieldsRemove total, failures"
  }
}
```

Three things to notice: `criteria` is a **list** (an SLO can carry more than one window); the SLI is a **DQL query** that must produce a timeseries field named `sli` — this one is SLO-02's availability query without `from:` / `interval:`, which the criteria supply; and `{{ .slo_target }}` is left **unquoted** so the rendered JSON carries a number. The shape matches Monaco's own `slo-v2` integration-test template.

> `--dry-run` **will not catch a bad SLI.** The template is valid JSON either way; a DQL error only surfaces when the SLO evaluates. Run the `indicator` query in a notebook first (add `, from:-7d` to the `timeseries` line there).

How References Work

Configurations reference each other with a **reference parameter** — `[project, configType, configId, property]`, or the long form below. For platform types `configType` is the type (`segment`, `slo-v2`, …); for Settings 2.0 it is the schema ID. To scope the SLO to a segment deployed from the same project:

```
configs:
  - id: checkout-availability
    config:
      name: "Checkout availability"
      template: checkout-availability.json
      parameters:
        slo_target: 99.0
        checkout_segment_id:
          type: reference
          configType: segment
          configId: checkout-production
          property: id
    type: slo-v2
```

…and add `"filterSegments": [{"id": "{{ .checkout_segment_id }}"}]` inside `customSli`. Monaco resolves the reference per environment at deploy time and deploys the segment first. (The `{{ .<type>.<id>.id }}` form is Monaco v1 syntax and does not load in v2.)

Classic — the alerting-profile version of this exercise

> **Dynatrace Classic — unupgraded tenants only.** Kept for estates that still maintain alerting profiles. *"Alerting profiles and problem notifications are Dynatrace Classic."* Delay (`delayInMinutes`) has no field on the successor object; it becomes the workflow trigger's **Minimum duration** option.
>
> **Sources:** Upgrade from Classic problem notification to simple workflows (DT docs) — *"The delay, update, and severity capabilities described in this guide exist only on the workflow trigger."*, Problem and event triggers (DT docs) — *"The Minimum duration option postpones the trigger until the problem has been open for at least the configured duration."*

`dynatrace-config/my-project/alerting-profiles/config.yaml`:

```
configs:
  - id: production-critical-alerts
    config:
      name: "Production Critical Alerts"
      template: production-critical-alerts.json
    type:
      settings:
        schema: builtin:alerting.profile
        scope: environment
```

Classic JSON template

Create `dynatrace-config/my-project/alerting-profiles/production-critical-alerts.json`:

```
{
  "name": "{{ .name }}",
  "severityRules": [
    {
      "severityLevel": "AVAILABILITY",
      "delayInMinutes": 0,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "ERRORS",
      "delayInMinutes": 5,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "PERFORMANCE",
      "delayInMinutes": 15,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    },
    {
      "severityLevel": "RESOURCE_CONTENTION",
      "delayInMinutes": 30,
      "tagFilterIncludeMode": "INCLUDE_ANY"
    }
  ]
}
```

Classic reference example

Configurations reference each other with a **reference parameter** — a list of `[project, configType, configId, property]`. For Settings 2.0 configs, `configType` is the schema ID:

```
configs:
  - id: notification-with-profile
    config:
      name: "Slack Notification"
      template: slack-notification.json
      parameters:
        alerting_profile_id: ["my-project", "builtin:alerting.profile",
"production-critical-alerts", "id"]
    type:
      settings:
        schema: builtin:problem.notifications
        scope: environment
```

Monaco resolves these references during deployment, ensuring the correct runtime IDs are used; the template reads the value as `{{ .alerting_profile_id }}`. (The `{{ .<type>.<id>.id }}` form is Monaco v1 syntax and does not load in v2.)

---

7. Validate Configuration

Always dry-run before deploying. `--dry-run` parses the manifest and every config, checks that templates are valid JSON, and resolves references — **without contacting the tenant**, so it cannot catch payload, schema or permission errors.

Run Dry-Run Validation

```
# Check file structure and references without deploying (no API calls)
monaco deploy --dry-run manifest.yaml
```

What Dry-Run Checks

Check	Description
YAML syntax	Valid YAML in `manifest.yaml` and `config.yaml` files
JSON syntax	Valid JSON in template files
Reference resolution	Reference parameters (`[project, configType, configId, property]`) point at configs that exist

What it does **not** check: the template against the Settings 2.0 schema, connectivity, or token scopes. The Monaco commands reference is explicit — *"A dry-run doesn't connect to Dynatrace and can't validate the content of the JSON sent to Dynatrace."* A deploy can still fail with HTTP 400 after a clean dry-run.

Common Errors

Error	Surfaces at	Cause	Fix
`template not found`	dry-run	Wrong path in `config.template`	Use path relative to `config.yaml`
`unresolved reference`	dry-run	Referenced config ID does not exist	Check spelling and project path
`invalid JSON`	dry-run	Syntax error in template	Run `python3 -m json.tool <file>` to find the error
`unknown schema`	deploy	Typo in schema ID, or a schema removed in Latest Dynatrace	Verify schema at Settings > Schema in Dynatrace
`insufficient permissions`	deploy	Token missing scopes	Add the missing scope to the platform token (platform types and Settings 2.0) or the access token (classic APIs)

---

8. Deploy Configuration

After validation passes, deploy to your target environment.

Deploy to All Environments

```
# Deploy to all environments defined in manifest.yaml
monaco deploy manifest.yaml
```

Deploy to a Specific Environment

```
# Deploy only to the dev environment
monaco deploy manifest.yaml --environment dev
```

Expected Output

Illustrative — exact log lines vary by Monaco version:

```
2026-04-04T10:00:00Z  INFO  Loading manifest "manifest.yaml"
2026-04-04T10:00:00Z  INFO  Projects to deploy: my-project
2026-04-04T10:00:01Z  INFO  Deploying config checkout-availability (slo-v2) to
dev...
2026-04-04T10:00:02Z  INFO  Deployed 1/1 configs successfully
```

Verify in Dynatrace

After deployment:

1. Open the **Service-Level Objectives** app
2. Confirm "Checkout availability" appears with the `lab:monaco` tag
3. Check that it evaluates — a status and error budget, not an error — and that the target reads 99 (the `slo_target` parameter)

Deploy a Specific Project

```
# Deploy only a specific project (useful when manifest has multiple projects)
monaco deploy manifest.yaml --project my-project
```

> **Tip:** Monaco uses the config `id` as an idempotency key. Re-deploying the same config updates it in place rather than creating duplicates.

---

9. Delete Configuration

Monaco can remove configurations that are no longer needed.

Create a delete.yaml File

Create `dynatrace-config/delete.yaml`:

```
delete:
  - project: my-project
    type: slo-v2
    id: checkout-availability
```

Execute Delete

The `type` is Monaco's config type (`slo-v2`) — or, for a Settings 2.0 object such as the classic alerting profile, the schema ID (`builtin:alerting.profile`). Non-classic entries need `project` and `id` together.

```
# Delete configurations listed in delete.yaml
# Note: the manifest is passed via --manifest, not as a positional argument
monaco delete --manifest manifest.yaml --file delete.yaml --environment dev
```

Delete Behavior

Behavior	Description
Scope	A `project` + `id` entry matches only an object Monaco created or onboarded. An `objectId` entry names one object directly, and a classic-API `name` entry matches whichever object has that name — including objects created in the UI

Behavior	Description
Not reversible	Re-deploying the config recreates it; it does not bring back the deleted object, so check anything that referenced the deleted object's ID
Environment-specific	Use `--environment` to target a single environment

> **Warning:** The `delete` command is irreversible for the target environment, and it has **no `--dry-run`**. Review `delete.yaml` in a pull request and run it against a non-production environment first (`--environment dev`).
>
> **Sources:** Monaco manifest and resources (DT docs) — *"Monaco can delete objects that are created by, and/or onboarded to, Monaco."*; delete command (Dynatrace GitHub) — flags `--manifest` , `--file` , `--group` , `--environment` only; classic deleter (Dynatrace GitHub) — resolves a `name` entry against every object the API returns.

---

10. Multi-Environment Promotion

A core Monaco strength is deploying the same configuration across dev, staging, and production with environment-specific overrides.

Multi-Environment Manifest

This manifest **replaces** the single-environment one from §3. It needs nine variables — `DT_<ENV>_URL` (the `.apps.` platform URL), `DT_<ENV>_TOKEN` and `DT_<ENV>_PLATFORM_TOKEN` for each of `DEV` , `STAGING` and `PROD` — and every command that touches an environment fails if one of that environment's variables is unset.

```yaml
manifestVersion: "1.0"

projects:
  - name: my-project
    path: my-project

environmentGroups:
  - name: default
    environments:
      - name: dev
        url:
          type: environment
          value: DT_DEV_URL
        auth:
          token:
            name: DT_DEV_TOKEN
          platformToken:
            name: DT_DEV_PLATFORM_TOKEN
      - name: staging
        url:
          type: environment
          value: DT_STAGING_URL
        auth:
          token:
            name: DT_STAGING_TOKEN
          platformToken:
            name: DT_STAGING_PLATFORM_TOKEN
      - name: prod
        url:
          type: environment
          value: DT_PROD_URL
        auth:
          token:
            name: DT_PROD_TOKEN
          platformToken:
            name: DT_PROD_PLATFORM_TOKEN
```

Environment-Specific Overrides

Use `environmentOverrides` (per environment) or `groupOverrides` (per environment group) in `config.yaml` to customize values. Both sit at the config level, as siblings of `config:`:

```
configs:
  - id: checkout-availability
    config:
      name: "Checkout availability"
      template: checkout-availability.json
      parameters:
        slo_target: 99.0
    type: slo-v2
    environmentOverrides:
      - environment: staging
        override:
          parameters:
            slo_target: 99.5
      - environment: prod
        override:
          parameters:
            slo_target: 99.9
```

Promotion Workflow

```
# Step 1: Deploy to dev and validate
monaco deploy manifest.yaml --environment dev

# Step 2: Deploy to staging after dev validation
monaco deploy manifest.yaml --environment staging

# Step 3: Deploy to production after staging validation
monaco deploy manifest.yaml --environment prod
```

Skip Configs in Certain Environments

`skip` is a boolean on `config:`. Default it to `false` and override it to `true` for the environments to exclude:

```
configs:
  - id: debug-dashboard
    config:
      name: "Debug Dashboard"
      template: debug-dashboard.json
      skip: false
    type:
      document:
        kind: dashboard
    environmentOverrides:
      - environment: prod
        override:
          skip: true
```

This deploys the debug dashboard to dev and staging but skips production entirely.

Validate Dashboard Configs Before Deploying Them

`monaco deploy --dry-run` is the pre-deploy gate. Run it before every real deploy; it resolves the manifest and references and checks template JSON without contacting the tenant, so a broken reference or an unresolved `type: environment` parameter with no `default` surfaces before it reaches a tenant.

For dashboard configs specifically, `--dry-run` is necessary but not sufficient: it validates the Monaco configuration around the template, not the dashboard document inside `template: <file>.json`. To Monaco that file is an opaque payload.

> **Forthcoming/rolling out (SaaS 1.344): a dashboard that fails validation will no longer load.** Published 07/27/2026, staged tenant rollout from 07/29/2026 — verify whether it has reached your tenant. Before 1.344 a failing dashboard still loaded and surfaced validation warnings; after, it does not load at all, and Dynatrace names **API- and AI-authored** dashboards as the most affected population — which is what a Monaco-templated dashboard is. So after a dry-run passes, deploy to a non-production environment and **open the dashboard in the Dashboards app to confirm zero validation warnings** before promoting to prod. Prefer exporting a working UI-authored dashboard as your template over hand-writing the tile map. Full gate: **DASH-07 §5**.

---

11. GitHub Actions CI/CD Pipeline

Automate Monaco deployments with a GitHub Actions workflow that validates on pull requests and deploys on merge to main.

Workflow File

Create `.github/workflows/monaco-deploy.yml`:

```
name: Deploy Dynatrace Config

on:
  push:
    branches: [main]
    paths: ['dynatrace-config/**']
  pull_request:
    branches: [main]
    paths: ['dynatrace-config/**']

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Validate (dry-run)
        run: monaco deploy --dry-run dynatrace-config/manifest.yaml --
environment dev
        env:
          DT_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DT_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DT_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}

  deploy:
    needs: validate
    if: github.ref == 'refs/heads/main' &amp;amp;&amp;amp; github.event_name == 'push'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/

      - name: Deploy to Dynatrace
        run: monaco deploy dynatrace-config/manifest.yaml --environment dev
```

```
        env:
          DT_ENV_URL: ${{ secrets.DT_ENV_URL }}
          DT_API_TOKEN: ${{ secrets.DT_API_TOKEN }}
          DT_PLATFORM_TOKEN: ${{ secrets.DT_PLATFORM_TOKEN }}
```

Workflow Explanation

Step	Trigger	What Happens
validate	Every PR and push to `main`	Installs Monaco, runs `--dry-run` to catch errors
deploy	Only on push to `main`	Installs Monaco, deploys all configurations to `dev`

The dry-run still needs the variables: Monaco resolves the manifest's environment variables when it loads the manifest, dry-run included, so a job without them fails with *environment-variable "…" was not found*.

Configure GitHub Secrets

Add these secrets to your GitHub repository (**Settings > Secrets and variables > Actions**):

Secret Name	Value
`DT_ENV_URL`	Your Dynatrace platform URL (e.g., `https://abc12345.apps.dynatrace.com`)
`DT_API_TOKEN`	Access token with the §2 access-token scopes
`DT_PLATFORM_TOKEN`	Platform token referenced by `auth.platformToken` in the manifest

Multi-Environment CI/CD

For promotion across environments, use the multi-environment manifest from §10 and replace the `jobs:` block above. Every job names its environment with `--environment` — without it Monaco deploys to **all** environments in the manifest, so a single `deploy` job would reach prod before staging — and every job sets that environment's three variables:

```
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/
      - name: Validate all environments (dry-run)
        run: monaco deploy --dry-run dynatrace-config/manifest.yaml --
environment dev,staging,prod
        env:
          DT_DEV_URL: ${{ secrets.DT_DEV_URL }}
          DT_DEV_TOKEN: ${{ secrets.DT_DEV_TOKEN }}
          DT_DEV_PLATFORM_TOKEN: ${{ secrets.DT_DEV_PLATFORM_TOKEN }}
          DT_STAGING_URL: ${{ secrets.DT_STAGING_URL }}
          DT_STAGING_TOKEN: ${{ secrets.DT_STAGING_TOKEN }}
          DT_STAGING_PLATFORM_TOKEN: ${{ secrets.DT_STAGING_PLATFORM_TOKEN }}
          DT_PROD_URL: ${{ secrets.DT_PROD_URL }}
          DT_PROD_TOKEN: ${{ secrets.DT_PROD_TOKEN }}
          DT_PROD_PLATFORM_TOKEN: ${{ secrets.DT_PROD_PLATFORM_TOKEN }}

  deploy-dev:
    needs: validate
    if: github.ref == 'refs/heads/main' &amp;amp;&amp;amp; github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: dev
    steps:
      - uses: actions/checkout@v7
      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/
      - name: Deploy to dev
        run: monaco deploy dynatrace-config/manifest.yaml --environment dev
        env:
          DT_DEV_URL: ${{ secrets.DT_DEV_URL }}
          DT_DEV_TOKEN: ${{ secrets.DT_DEV_TOKEN }}
          DT_DEV_PLATFORM_TOKEN: ${{ secrets.DT_DEV_PLATFORM_TOKEN }}

  deploy-staging:
    needs: deploy-dev
```

```
    if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - uses: actions/checkout@v7
      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/
      - name: Deploy to staging
        run: monaco deploy dynatrace-config/manifest.yaml --environment
staging
        env:
          DT_STAGING_URL: ${{ secrets.DT_STAGING_URL }}
          DT_STAGING_TOKEN: ${{ secrets.DT_STAGING_TOKEN }}
          DT_STAGING_PLATFORM_TOKEN: ${{ secrets.DT_STAGING_PLATFORM_TOKEN }}

  deploy-prod:
    needs: deploy-staging
    if: github.ref == 'refs/heads/main' &amp;&amp; github.event_name == 'push'
    runs-on: ubuntu-latest
    environment: prod
    steps:
      - uses: actions/checkout@v7
      - name: Install Monaco
        run: |
          curl -L https://github.com/Dynatrace/dynatrace-configuration-as-
code/releases/latest/download/monaco-linux-amd64 -o monaco
          chmod +x monaco
          sudo mv monaco /usr/local/bin/
      - name: Deploy to prod
        run: monaco deploy dynatrace-config/manifest.yaml --environment prod
        env:
          DT_PROD_URL: ${{ secrets.DT_PROD_URL }}
          DT_PROD_TOKEN: ${{ secrets.DT_PROD_TOKEN }}
          DT_PROD_PLATFORM_TOKEN: ${{ secrets.DT_PROD_PLATFORM_TOKEN }}
```

Add the nine `DT_<ENV>_URL` / `DT_<ENV>_TOKEN` / `DT_<ENV>_PLATFORM_TOKEN` secrets. Each deploy job waits for the previous one, so prod only runs after staging succeeds.

> **Sources:** Monaco commands reference (DT docs) — *"If neither --group nor --environment is present, all environments are used."*; manifest loader (Dynatrace GitHub) — `environment-variable %q`

> `was not found`, `requested environment %q not found`.

Use GitHub Environments with required reviewers to gate production deployments.

---

12. Summary & Checklist

Deployment Checklist

- [ ] Monaco CLI installed and `monaco version` returns output
- [ ] Platform token created with the §2 scopes (`app-engine:apps:run`, the three `settings:` scopes and the three `slo:` scopes), plus an access token with `DataExport`
- [ ] `DT_ENV_URL` set to the platform (`.apps.`) URL
- [ ] `manifest.yaml` created with correct environment variable references
- [ ] Downloaded existing configuration with `monaco download`
- [ ] Explored directory structure and understood `config.yaml` format
- [ ] Created a new configuration (an `slo-v2` SLO with a DQL SLI) from scratch
- [ ] Validated with `monaco deploy --dry-run` (no errors)
- [ ] Deployed to target environment with `monaco deploy`
- [ ] Verified deployed config in Dynatrace UI
- [ ] Tested delete workflow with `delete.yaml`
- [ ] Configured multi-environment promotion with overrides
- [ ] Created GitHub Actions workflow for CI/CD

Key Takeaways

Concept	Summary
Manifest	Entry point defining projects and environments
Config Types	`slo-v2`, `segment`, `automation`, `document`, `bucket`, `settings`, `api` (classic) — platform types need `auth.platformToken` or `auth.oAuth`
Download	Export existing tenant config to versioned files

Concept	Summary
Dry-Run	Always dry-run before deploying — structure and references only, no API calls
Deploy	Idempotent -- re-running updates in place
Delete	Removes exactly the entries in `delete.yaml` — no dry-run, not reversible
Overrides	`environmentOverrides` / `groupOverrides` for per-environment values and `skip`
CI/CD	Validate on PR, deploy on merge to main

References

- Monaco Documentation
- Monaco GitHub Repository
- Monaco API support and access handling (DT docs)
- Settings Schema Reference
- Monaco commands reference (DT docs)
- Monaco configuration YAML reference (DT docs)
- Settings 2.0 schemas that are removed in Latest Dynatrace (DT docs)

---

*Return to **AUTOM-03: Monaco Configuration-as-Code** for the lecture content, or continue to **AUTOM-04: Terraform Provider**.*

Community Resources

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.30.0 at time of writing, released 09/23/2026)
community-examples / configuration-as-code	9 starter templates in `basic-templates-monaco`, plus pipeline observability samples (moved here from the archived `dynatrace-configuration-as-code-samples` repo)
easytrade	Real-world Monaco project structure with manifest.yaml

Repository	Description
Dynatrace-Config-Manager	**Archived** — historical reference only
monaco-self-paced-exercises	6 structured exercises (install, auto-tag, download, variables, delete/restore, linking)
monaco-demo	Working GitHub Actions workflow for Monaco deploy

Community Resources & Examples

After completing the hands-on steps above, these repositories provide deeper examples and exercises to reinforce what you've practiced:

Official Repositories

Repository	Description
dynatrace-configuration-as-code	Official Monaco CLI (v2.30.0 at time of writing, released 09/23/2026) — Go binary, Apache-2.0
community-examples / configuration-as-code	Current home of the Monaco samples, including 9 starter templates in `basic-templates-monaco` (mostly schemas removed in Latest Dynatrace — syntax examples, not templates for a new tenant). The old `dynatrace-configuration-as-code-samples` repo is archived: *"The Dynatrace Configuration as Code samples are no longer maintained here."*
easytrade	Demo microservices app with a working `monaco/` directory (manifest.yaml, detection rules, workflows)
Dynatrace-Config-Manager	**Archived** — *"This repository has been archived and is no longer maintained."* Historical reference only

Training & Exercises (recommended next)

Repository	Description
monaco-self-paced-exercises	6 structured exercises: install, auto-tag, download, variables, delete/restore, linking configs
monaco-demo	Working GitHub Actions workflow for Monaco deploy with "crawl-walk-run" adoption methodology

Pipeline Observability Samples

Monaco configurations for ingesting CI/CD pipeline events via OpenPipeline, under `community-examples/configuration-as-code/`:

Directory	CI/CD Platform
`github_pipeline_observability`	GitHub Actions
`gitlab_pipeline_observability`	GitLab CI
`azure_devops_observability`	Azure DevOps
`argocd_observability`	ArgoCD

> **Note:** Monaco v1 (`dynatrace-oss/dynatrace-monitoring-as-code`) is no longer supported. Current Monaco releases have no conversion command: *"The last Monaco version to support the convert command is Monaco version 2.19.0."* Either run Monaco 2.19.0's `convert` per the migrating to v2 guide (DT docs), or re-run `monaco download` against the source tenant.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.